3.15.3 Considerations when deploying to a subdirectory

Deploying to a subdirectory in production has implications on various parts of Rails.

- development environment:
- testing environment:
- serving static assets:
- asset pipeline:

4 Rails Environment Settings

Some parts of Rails can also be configured externally by supplying environment variables. The following environment variables are recognized by various parts of Rails:

- `ENV["RAILS_ENV"]` defines the Rails environment (production, development, test, and so on) that Rails will run under.
- `ENV["RAILS_RELATIVE_URL_ROOT"]` is used by the routing code to recognize URLs when you deploy your application to a subdirectory.
- `ENV["RAILS_CACHE_ID"]` and `ENV["RAILS_APP_VERSION"]` are used to generate expanded cache keys in Rails' caching code. This allows you to have multiple separate caches from the same application.

5 Using Initializer Files

After loading the framework and any gems in your application, Rails turns to loading initializers. An initializer is any Ruby file stored under `config/initializers` in your application. You can use initializers to hold configuration settings that should be made after all of the frameworks and gems are loaded, such as options to configure settings for these parts.
You can use subfolders to organize your initializers if you like, because Rails will look into the whole file hierarchy from the initializers folder on down.

If you have any ordering dependency in your initializers, you can control the load order through naming. Initializer files are loaded in alphabetical order by their path. For example, `01_critical.rb` will be loaded before `02_normal.rb`.

6 Initialization events

Rails has 5 initialization events which can be hooked into (listed in the order that they are run):

- `before_configuration`: This is run as soon as the application constant inherits from `Rails::Application`. The `config` calls are evaluated before this happens.
- `before_initialize`: This is run directly before the initialization process of the application occurs with the `:bootstrap_hook` initializer near the beginning of the Rails initialization process.
- `to_prepare`: Run after the initializers are run for all Railties (including the application itself), but before eager loading and the middleware stack is built. More importantly, will run upon every request in `development`, but only once (during boot-up) in `production` and `test`.
- `before_eager_load`: This is run directly before eager loading occurs, which is the default behavior for the `production` environment and not for the `development` environment.
- `after_initialize`: Run directly after the initialization of the application, after the application initializers in `config/initializers` are run.